

Python Topic 10 — Functions

Full Stack Python + AI Roadmap • Taught in 3 layers: New to Python → Intermediate → Expert



Layer 1 — New to Python

A **function** is a reusable block of code you give a name, so you can run it whenever you want.

```
def greet():  
    print("Hello!")  
  
greet()      # Hello! → "call" the function to run it
```

You define with `def`, a name, parentheses, and a colon `:` — then the indented body.

Functions can take **inputs** (parameters) and give back **outputs** (return):

```
def add(a, b):  
    return a + b  
  
result = add(3, 5)      # result = 8
```

Note: `return` sends a value back to whoever called the function. Without `return`, a function gives back `None`.



Layer 2 — Intermediate (real code + how it works)

Syntax vs JavaScript

```
// JS  
function add(a, b) {  
    return a + b;  
}  
const multiply = (a, b) => a * b;  
  
# Python  
def add(a, b):  
    return a + b
```

Key differences: `def` keyword, colon + indentation (no braces), no `function` /arrow distinction.

Default parameters

```
def greet(name, greeting="Hello"):
    return f"{greeting}, {name}!"

greet("Akshay")           # "Hello, Akshay!"
greet("Akshay", "Hi")     # "Hi, Akshay!"
```

Keyword arguments — pass by name, any order

```
def create_user(name, age, city):
    return f"{name}, {age}, {city}"

create_user(age=26, name="Akshay", city="Pune")  # order doesn't matter!
```

Makes function calls self-documenting. Very common in Python libraries.

***args** — any number of positional arguments (collects into a tuple)

```
def total(*numbers):
    return sum(numbers)

total(1, 2, 3)           # 6
total(1, 2, 3, 4, 5)     # 15
```

****kwargs** — any number of keyword arguments (collects into a dict)

```
def build_profile(**info):
    return info

build_profile(name="Akshay", role="dev")  # {'name': 'Akshay', 'role': 'dev'}
```

i JS analogy: ***args** ≈ JS rest params **...args** ; ****kwargs** ≈ collecting named options into an object.

Returning multiple values (recall tuples, Topic 6)

```
def min_max(nums):
    return min(nums), max(nums)  # returns a tuple

low, high = min_max([3, 7, 1])  # unpacked: low=1, high=7
```

Lambda — a small anonymous one-line function

```

square = lambda x: x ** 2
square(5)      # 25

# most common use – as a sort/map key
words = ["python", "ai", "rag", "javascript", "go"]
words.sort(key=lambda w: len(w))      # sort by length (shortest first)
# words → ["ai", "go", "rag", "python", "javascript"]
#           2     2     3         6         10   ← lengths

```

✓ **Tip:** `key=lambda w: len(w)` tells `.sort()` to compare by each word's length instead of alphabetically. You could shorten it to `key=len` since `len` is itself a function.

Lambdas are like JS arrow functions but limited to a single expression (no statements, no multiple lines).

Layer 3 — Expert (internals & gotchas)

1. The mutable default argument trap (the classic)

```

def add_item(item, lst=[]):      # 🚩 the [] is created ONCE, shared!
    lst.append(item)
    return lst

add_item(1)      # [1]
add_item(2)      # [1, 2] ← not [2]!

```

🚩 Fix with None sentinel:

```

def add_item(item, lst=None):
    if lst is None:
        lst = []
    lst.append(item)
    return lst

```

The default is evaluated once at *definition* time, not per call. This is the #1 Python interview gotcha.

2. Scope — the LEGB rule (Local → Enclosing → Global → Built-in)

```

x = "global"

def outer():
    x = "enclosing"
    def inner():
        x = "local"

```

```

    print(x)      # "local" – checks Local first
    inner()       # call inner so it runs

outer()          # prints "local"
# Python searches: Local → Enclosing → Global → Built-in, stops at first match

```

To modify an outer variable, you need `global` or `nonlocal`:

```

count = 0

def increment():
    global count      # without this, count += 1 raises an error
    count += 1

def outer():
    x = 0
    def inner():
        nonlocal x    # modifies the enclosing x, not a new local
        x += 1
    inner()

```

⚠ **Key rule:** reading a variable searches $L \rightarrow E \rightarrow G \rightarrow B$, but **assigning** (`x = ...`) always creates a new local by default. Use `global` / `nonlocal` to rebind an outer name.

3. Positional-only and keyword-only arguments

```

def func(a, b, /, c, d, *, e, f):
    ...

# a, b → positional ONLY (before /)
# e, f → keyword ONLY (after *)
# c, d → either

```

You'll see `/` and `*` in library signatures — they enforce how args must be passed.

4. Unpacking arguments with `*` and `**`

```

def add(a, b, c):
    return a + b + c

nums = [1, 2, 3]
add(*nums)      # unpacks list → add(1, 2, 3)

opts = {"a": 1, "b": 2, "c": 3}
add(**opts)     # unpacks dict → add(a=1, b=2, c=3)

```

The reverse of `*args` / `**kwargs` — spreading a collection *into* a call (like JS spread `add(...nums)`).

5. Functions are first-class objects

```
def apply(func, value):
    return func(value)

apply(square, 5)          # 25 – pass a function as an argument

# store in variables, lists, dicts
operations = {"double": lambda x: x*2, "square": lambda x: x**2}
operations["double"](10)  # 20
```

This underpins decorators, callbacks, and functional patterns (map/filter/sorted keys).

6. Type hints (essential for FastAPI/production)

```
def greet(name: str, age: int = 0) -> str:
    return f"{name} is {age}"
```

Hints don't enforce types at runtime, but power IDE autocomplete, type checkers (mypy), and frameworks like FastAPI/Pydantic (which *do* validate from them). You'll use these everywhere in real code.

7. Docstrings — documentation as the first string

```
def add(a, b):
    """Return the sum of a and b."""
    return a + b

add.__doc__          # "Return the sum of a and b."
help(add)            # shows the docstring
```

Interview Q&A Cards

Q: Why is a mutable default argument dangerous?

A: It's created once at definition time and shared across all calls, so state leaks between invocations. Use `None` as a sentinel and build the object inside the function.

Q: Explain LEGB scope.

A: Name lookup order — Local, Enclosing, Global, Built-in — stopping at the first match. Assignment creates a local by default; `global` / `nonlocal` rebind outer names.

Q: `*args` vs `**kwargs` ?

A: `*args` collects extra positional arguments into a tuple; `**kwargs` collects extra keyword arguments into a dict. Use `*` / `**` at call sites to unpack collections into arguments.

Q: What does "functions are first-class" mean?

A: Functions are objects — you can assign them to variables, pass them as arguments, return them, and store them in collections. This enables decorators, callbacks, and higher-order functions.

🔗 **Memory hook:** `def` + colon, `return` or you get `None` . `*args` collects positionals, `**kwargs` collects named. Never default to `[]` — default to `None` .

✅ **One-liner for interviews:** "Functions use `def` , support defaults, keyword args, `*args` / `**kwargs` . Scope follows LEGB; use `global` / `nonlocal` to rebind outer names. Never use a mutable default — it's shared across calls. Functions are first-class objects, enabling decorators and callbacks. Type hints power tooling and frameworks like FastAPI."